SmartFin

A personal finance copilot for Thailand that reads your PromptPay slips and reasons over your own spending — instead of asking you to type it in.

Thaw Zin Moe Myint
Student

Flutter · Go · MongoDB · Gemini
Stack

Alpha Test
Milestone

Thai users already have a record of every payment. It is trapped in their photo gallery.

1 · MANUAL ENTRY FAILS

Budgeting apps depend on the user typing each expense. People abandon this within weeks, so the data is incomplete exactly when advice would matter.

2 · NO BANK AGGREGATION

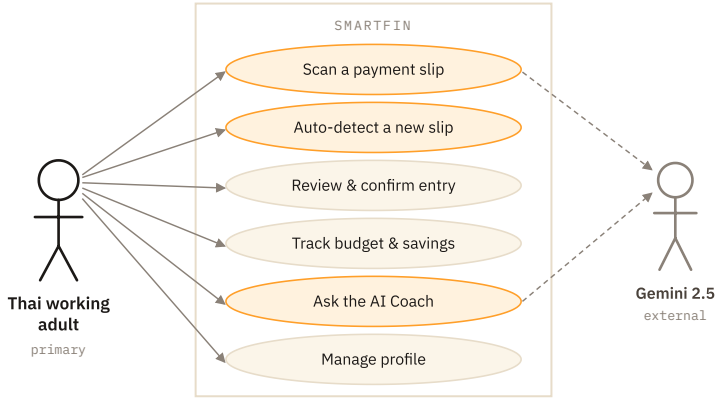
Thailand has no consumer open-banking API. Services like Plaid do not cover Thai retail banks, so automatic transaction import is not available to a student project.

3 · GENERIC ADVICE

Chatbot finance tools answer from general knowledge. They cannot say what *this* user spent on food this month, because they never see the data.

The opening. Every PromptPay, K PLUS, SCB EASY, Krungthai NEXT and Bangkok Bank transfer ends in a confirmation slip the user screenshots. The record already exists — it is simply an image. SmartFin turns that image into structured data, then lets an AI agent reason over it.

Who it is for, and what they can do



Highlighted ellipses are the differentiating use cases — the demo follows them.

PRIMARY USER

A Thai working adult or student who pays by **PromptPay several times a day**, keeps every confirmation slip in their gallery, and has abandoned at least one budgeting app.

DESIGN CONSEQUENCES

- Currency is **integer Thai Baht (฿)** end-to-end — no floats, no decimals.
- Slips print Buddhist-era years: 2569 → 2026 conversion is required, not optional.
- Merchant names arrive in Thai script and are transliterated on extraction.
- Month boundaries are pinned to `UTC+07:00`, not server time.

What the project commits to — and what it deliberately excludes

IN SCOPE

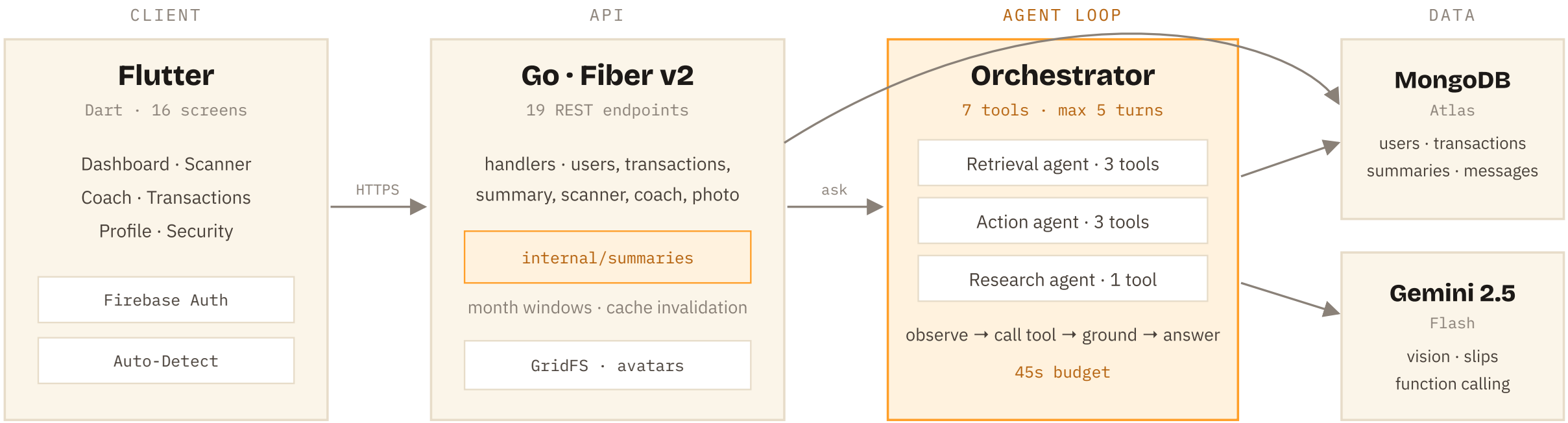
- **Slip & receipt capture** — manual scan and Auto-Detect, both extracted by a vision model.
- **Transaction ledger** — create, list, edit, delete; manual entry as a fallback path.
- **Monthly budgeting** — income, budget limit, category breakdown, per-month summaries.
- **Agentic AI Coach** — answers grounded in the user's own records, and able to act on them.
- **Savings goals** — set by the user or by the coach, with progress derived from actual months.
- **Account management** — profile, photo, credentials, account deletion.

OUT OF SCOPE

- **Direct bank integration** — no consumer open-banking API exists in Thailand; the slip *is* the integration.
- **Moving money** — SmartFin never initiates a payment or transfer. It is read-and-advise only.
- **Investment or tax advice** — the coach redirects to general financial health.
- **Multi-currency** — Thai Baht only, which is what makes integer money safe.
- **Multi-user or shared budgets** — one account, one person.
- **iOS release build** — Android is the delivery target for this milestone.

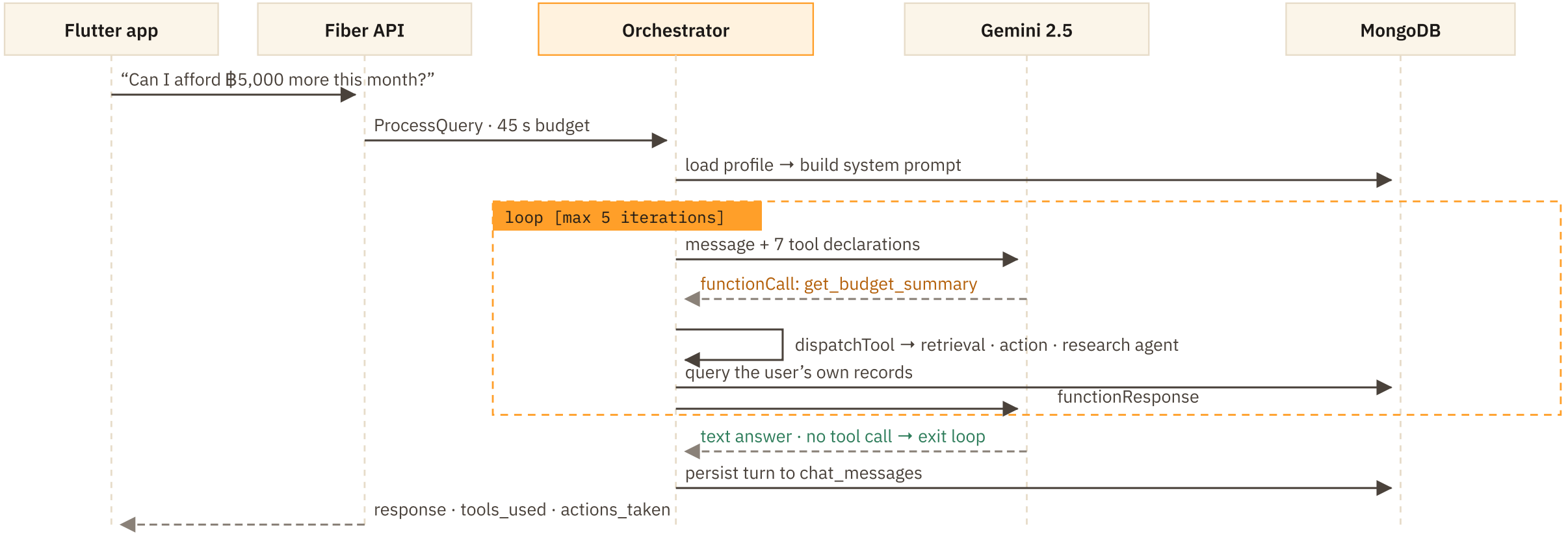
Why the boundary sits here. Every excluded item either requires a commercial banking agreement or moves the project from *advice* to *financial transactions*, which carries a regulatory burden a one-semester project cannot honestly meet.

Three tiers, one agent loop



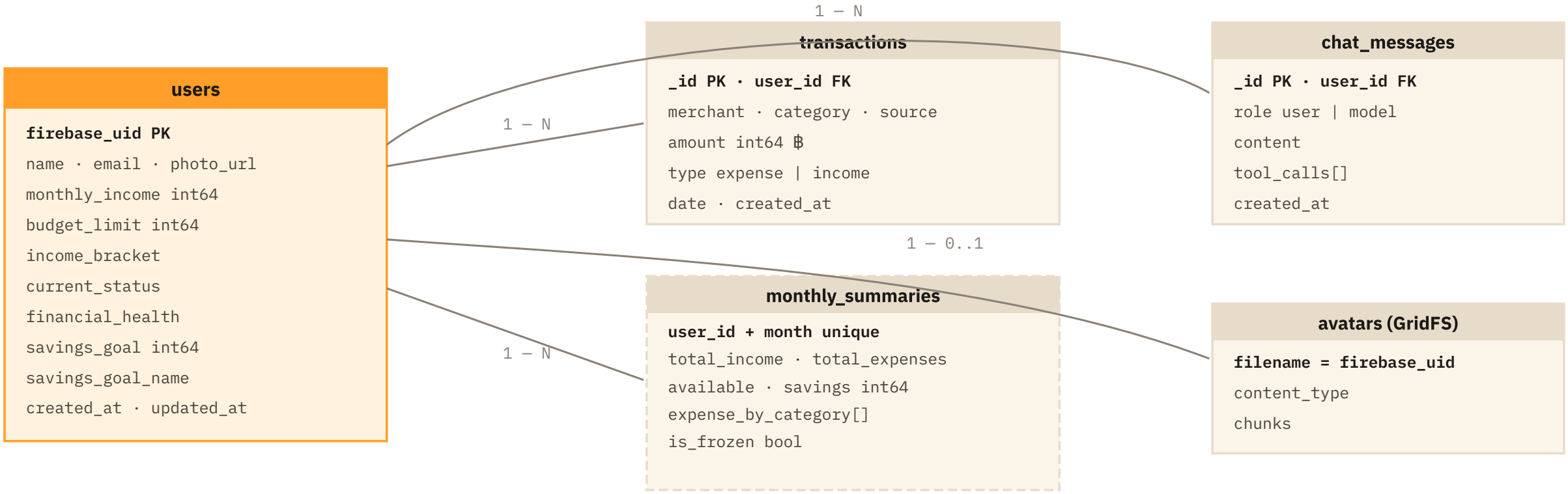
The coach never queries Mongo directly — every read and write goes through a declared tool, which is what makes its actions auditable.

How one question becomes a grounded answer



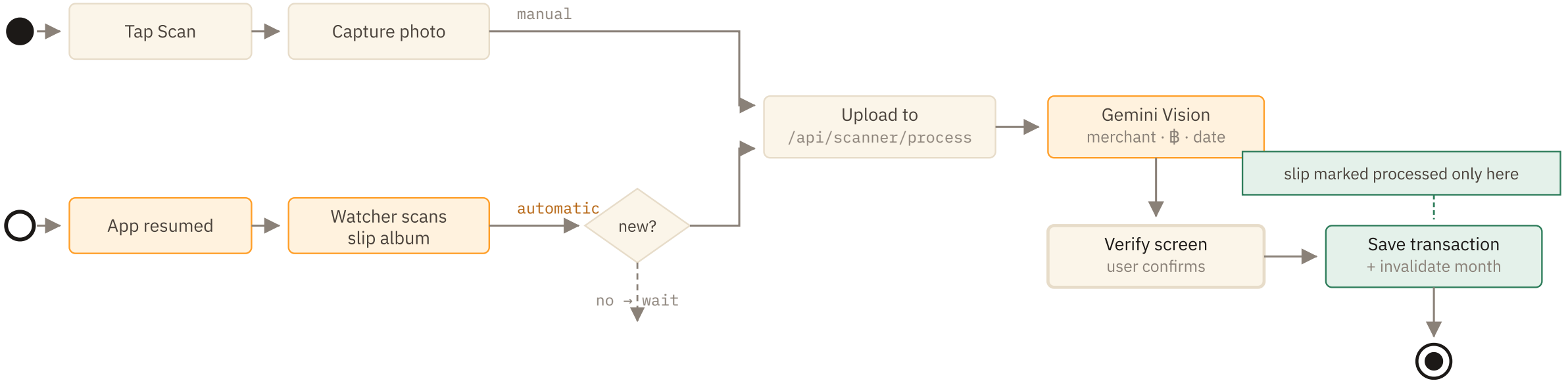
The model is never given database access — it can only request one of seven declared tools, which is what makes every action it takes reviewable.

Four collections, keyed by one identity



Every collection is keyed by the Firebase UID *string*, not a Mongo ObjectID. **monthly_summaries** is drawn dashed because it is derived — a cache recomputed from transactions and invalidated on every write, never a source of truth.

Two ways in, one path to a saved transaction



Extraction persists nothing on its own — the user's confirmation is what writes the transaction, and only then is the gallery slip retired.

Functional requirements

11 / 14 REQUIREMENTS COMPLETE	**19** REST ENDPOINTS
16 APP SCREENS	**~15.1k** LINES OF CODE

REQUIREMENT	EVIDENCE IN THE BUILD	STATUS
FR-1 · Register & authenticate	Firebase email/password and Google sign-in	● DONE
FR-2 · Capture a slip by camera	Scanner → processing → verify flow	● DONE
FR-3 · Extract merchant, amount, date	Gemini Vision, temperature 0, Thai + Buddhist-era handling	● DONE
FR-4 · Auto-Detect slips	Album cascade, dedup, user-toggleable	● DONE
FR-5 · Confirm before saving	Verify screen; extraction persists nothing on its own	● DONE
FR-6 · Manage transactions	Create, list, edit, delete; manual entry path	● DONE
FR-7 · Monthly budget summary	Income, expenses, available, category breakdown	● DONE
FR-8 · Ask a grounded question	Agent loop over the user's own records	● DONE
FR-9 · Coach performs actions	Sets goals, updates budget, adds transactions	● DONE
FR-10 · Savings goal + progress	Shared by the profile UI and the agent; dashboard tracker	● DONE

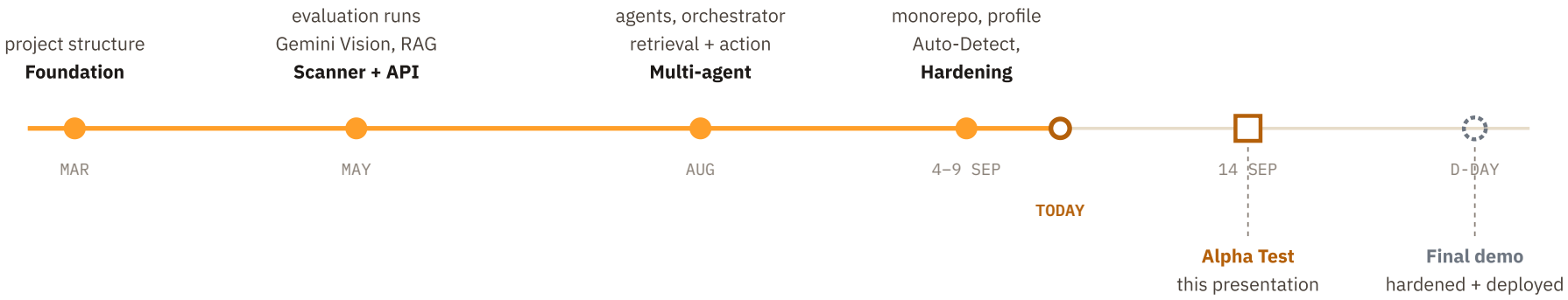
Non-functional requirements

QUALITY	TARGET AND HOW IT IS MET	STATUS
Performance	Coach replies within a `45 s` server budget; client waits `60 s` so it never aborts a request the server finished. Images downscaled to 800 px before upload. Past months served from a frozen cache.	MET
Correctness	Money is `int64` Baht end-to-end. Month windows anchored to `UTC+07:00` so a 06:00 Bangkok expense is not filed in the previous month. Every write invalidates the affected summary cache.	MET
Usability	Capture is two taps from a detected slip. Amounts are pre-formatted server-side. Failures state a cause and a next step rather than a generic error.	MET
Reliability	A failed scan never silently loses a receipt — the slip is marked processed only after its transaction saves. A failed dashboard load renders an explicit error state, never a fabricated ฿0.	MET
Security	Firebase identity; re-authentication required before password change or account deletion; single-transaction routes scoped to their owner. **Gap:** the API does not yet verify Firebase ID tokens server-side.	PARTIAL
Privacy	Account deletion erases the profile, transactions, summaries and coach conversations in one operation. Slip images are processed and not retained by the scanner.	MET
Maintainability	Month rules and cache invalidation live in one shared package used by both handlers and agents. Profile updates bind to a typed struct rather than a free-form map.	MET
Testability	13 unit tests cover month-boundary maths, partial-update binding and the agent tool contract. Widget and integration tests are not yet written.	PARTIAL

Reported honestly. Two qualities are partial. Both are named on the remaining-work slide with a date, rather than being presented as complete.

TIMELINE

Past progress, current status, and the road to D-Day



COMPLETED

Capture pipeline, ledger, budgeting, the full agent loop, and account management — 11 of 14 functional requirements.

CURRENT STATUS

Feature-complete for the alpha demo. Both halves build clean; the app runs against a local API on an Android emulator.

REMAINING

Server-side token verification, deployment, wider test coverage, and the last two screens.

Demo flow — one continuous story, following the use cases

1. **Sign in and set up the profile.** Income bracket, status and financial health — the same values the coach is later grounded in. `UC: Manage profile`
2. **Pay by PromptPay, return to the app.** Auto-Detect offers the new slip unprompted. `UC: Auto-detect new slips`
3. **Confirm the extracted entry.** Merchant, ฿ amount and date come back from the vision model; the user corrects the category and saves. `UC: Review & confirm`
4. **Watch the dashboard react.** Budget ring, category breakdown and savings tracker all move on the same figure. `UC: Track budget`
5. **Ask the coach a real question.** “Can I afford ฿5,000 more this month?” — it calls its own tools and answers from the ledger. `UC: Ask the AI Coach`
6. **Let the coach act.** “Set aside ฿50,000 for a Japan trip.” The goal is written, and the dashboard tracker updates. `UC: Ask the AI Coach`

WHAT THE FLOW IS DESIGNED TO PROVE

- The user **never types an amount** from step 2 onward.
- The coach reads **this user's records**, not general knowledge.
- The coach **writes** as well as reads — step 6 changes stored state.
- One figure moves consistently through capture, budget and goal.

FALLBACK IF THE NETWORK FAILS

A seeded demo account with six months of Thai transactions is loaded in advance, and a sample slip image ships with the app — so steps 3 to 6 can be shown without a live

What we found when we audited the build

Before this milestone the codebase was reviewed end to end — backend logic, client behaviour, and the contract between them. The defects worth reporting were not in the AI features; they were in the arithmetic underneath them.

REPRESENTATIVE DEFECTS, NOW FIXED

- **Stale monthly cache.** The refreshed summary was never written back, so the dashboard and the coach could report different totals for the same month.
- **Timezone freeze.** A month could be frozen at ฿0 when the phone and a UTC server disagreed about the date at midnight.
- **Backdated entries invisible.** Adding a forgotten expense to a closed month never invalidated that month's totals.
- **Receipts lost on failure.** A slip was marked processed *before* the scan ran, so a failed scan hid it permanently.

HOW THEY WERE PREVENTED FROM RECURRING

- Month boundaries and cache invalidation moved into `internal/summaries` — one implementation, used by handlers and agents alike.
- 13 unit tests now pin the month arithmetic, the partial-update contract and the agent's tool list.
- Profile updates bind to a typed request struct, so an unknown field can no longer be written into the user document.
- Single-transaction routes are scoped by owner, closing an ID-guessing path.

Why this belongs in the presentation. The AI layer is the visible part of the project, but its answers are only as trustworthy as the ledger beneath it. A coach that confidently quotes a stale number is worse than no coach at all.

REMAINING WORK

From alpha to final demonstration

TASK	WHY IT MATTERS	PRIORITY
Verify Firebase ID tokens server-side	The API currently trusts the UID in the request. This is the single most important correctness-of-access gap, and every endpoint depends on the same middleware.	CRITICAL
Deploy the API	The client still points at a hardcoded LAN address. A hosted API removes the demo's dependence on a laptop on the same network.	HIGH
Rotate exposed credentials	An example configuration file and an old evaluation script contain real-looking keys recoverable from history.	HIGH
Transaction detail & filtering	Completes FR-12 and FR-13; the detail endpoint already exists and is unused.	MEDIUM
Widget & integration tests	Client behaviour is currently verified by hand. The scan pipeline in particular deserves an automated path.	MEDIUM

NEXT 2 WEEKS

Auth middleware, deployment, credential rotation.

FOLLOWING SPRINT

FR-12 and FR-13, then the test suite.

BEFORE D-DAY

Evaluation harness re-run, full demo rehearsal on the hosted API.

SUMMARY

Where SmartFin stands today

11 / 14 FUNCTIONAL REQUIREMENTS	**6 / 8** NON-FUNCTIONAL MET
7 AGENT TOOLS LIVE	**2** CAPTURE PATHS

THE CLAIM BEING DEMONSTRATED

A Thai user can go from paying by PromptPay to seeing that payment reflected in their budget, their category breakdown and their savings goal — **without typing a single number** — and can then ask an agent that reads and writes their real records what to do about it.

KNOWN LIMITATIONS, STATED PLAINLY

- The API does not yet verify identity tokens.
- The backend runs locally; it is not deployed.
- Client test coverage is manual.
- Two secondary screens remain unbuilt.

Questions